

Implementation and Evaluation of a Lossy Spike Compression Framework for Spiking Neural Networks

Satvik GANESH (s1302003)

Supervisor: Khanh N. Dang

School of Computer Science and Engineering

University of Aizu, Fukushima, Japan

Date: August 10th 2026

Content

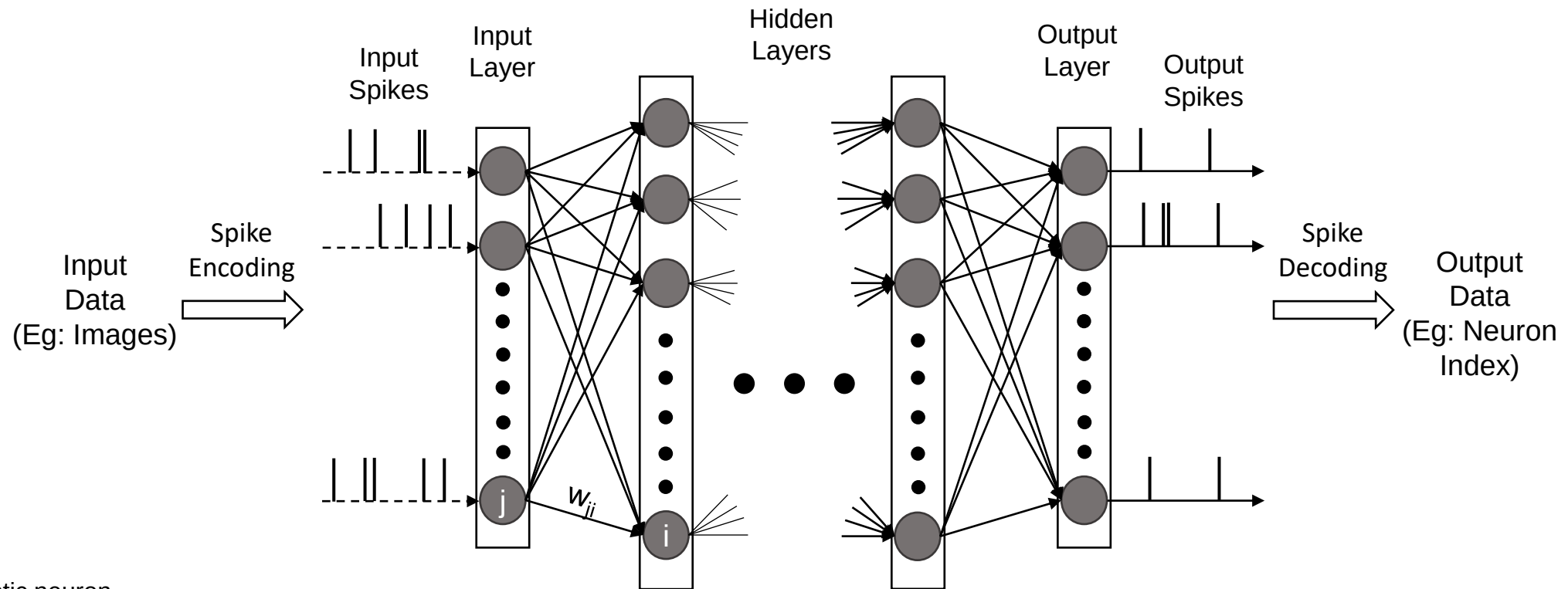
- Background
- Motivation
- Proposed Method
- Results
- Conclusion

Content

- Background
 - Spiking Neural Networks
 - LIF Neuron
 - Rate Coding
 - AER
 - Pattern Mining Algorithms
- Motivation
- Proposed Method
- Results
- Conclusion

Spiking Neural Networks (SNNs)

- Designed to mimic biological neurons, use spikes to convey information [1]



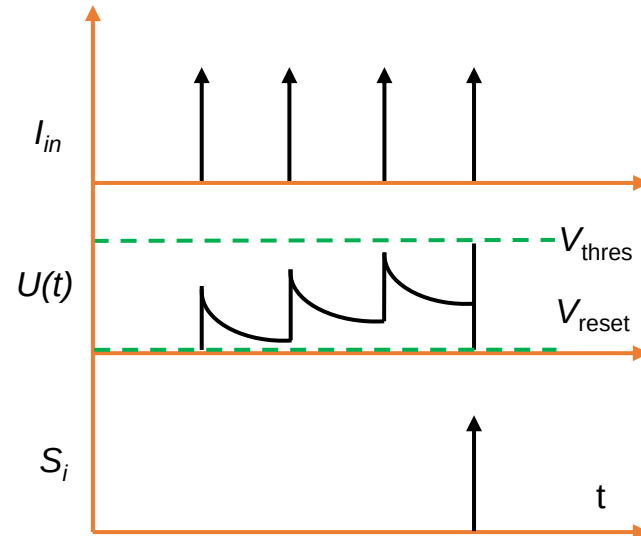
j: Pre-synaptic neuron
i: Post-synaptic neuron
 w_{ji} : Synaptic weight between i and j

Fig. 1: Structure of SNN

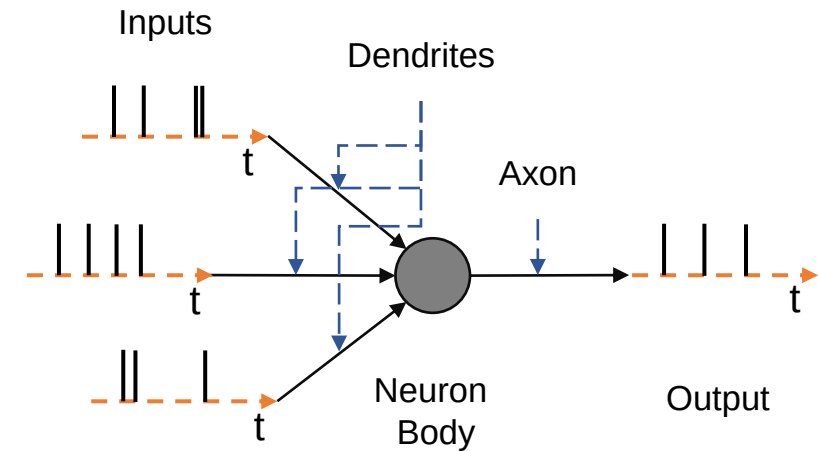
Leaky Integrate and Fire (LIF) Neuron

τ : Time constant
 $U(t)$: Membrane potential
 R : Membrane resistance
 I_{in} : Input current

S_i : Output spike
 V_{thres} : Threshold potential
 V_{reset} : Reset potential



(a)



(b)

LIF equation:

$$\tau \frac{dU(t)}{dt} = -U(t) + RI_{in}(t)$$

Fig. 2: (a) LIF potential dynamics (b) A typical SNN neuron

Rate Coding

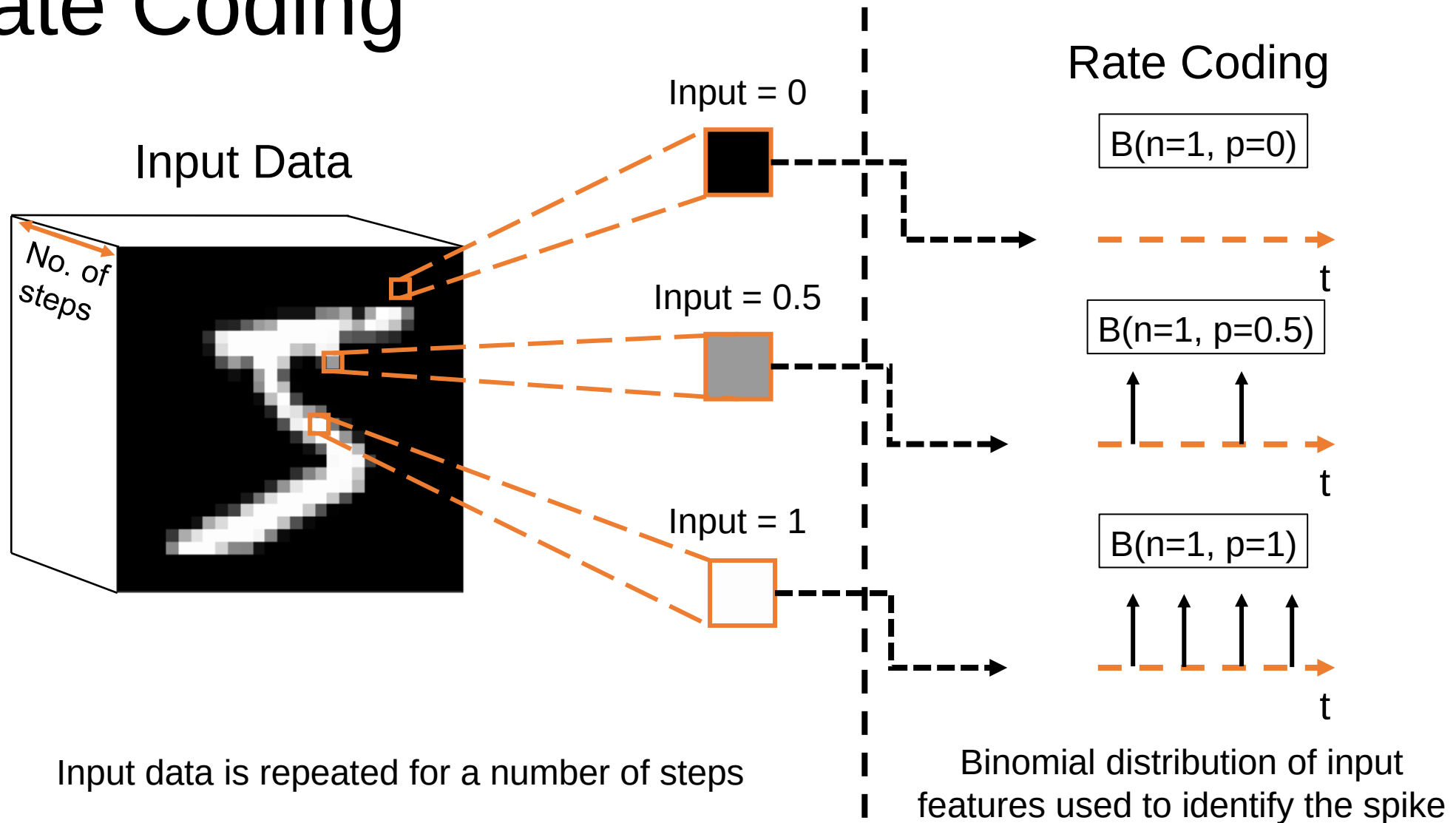


Fig. 3: Rate Coding for SNNs

Address Event Representation

- Neuromorphic protocol encoding timing and location of spike events [2]
- Only conveys the timing and location of active spikes

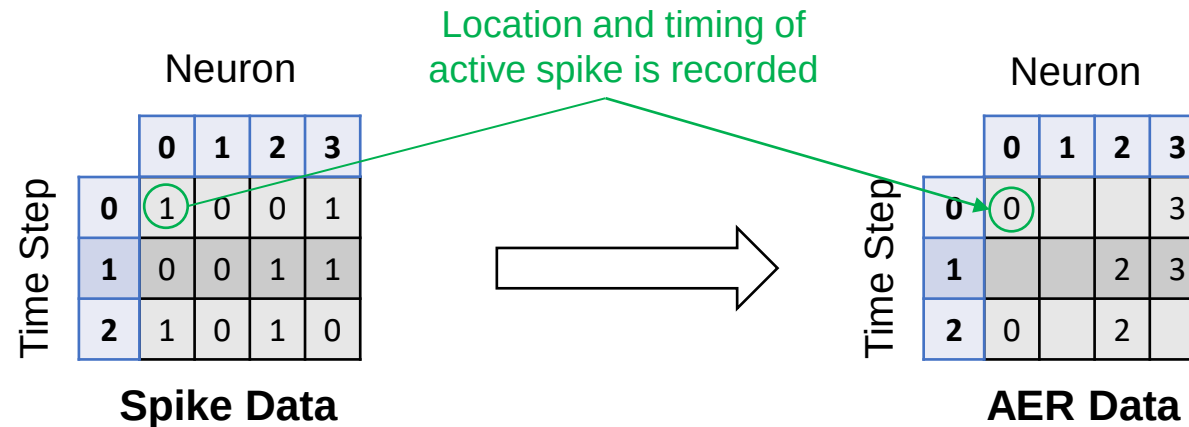


Fig. 4: Spike to AER conversion

Pattern Mining Algorithms

- Find interesting and relevant patterns in dataset
- **FP-Growth:** Builds an FP-Tree to directly extract frequent patterns [3]
- **FP-Max:** Uses an FP-Tree with pruning to find only maximal frequent patterns [4]

Content

- Background
- **Motivation**
- Proposed Method
- Results
- Conclusion

Motivation

- Rate coding: Simple & noise-resilient, but higher spikes = higher energy & latency
- Limits scalability in resource-constrained systems

Observation during Rate Coding

- Neurons fire in groups at the same timestep (Fig. 5)
- Highly similar spike patterns observed at timesteps 4, 9, 18 in hidden layer
- $\sim 30.50\%$ spike similarity (Jaccard), so moderate–high temporal redundancy

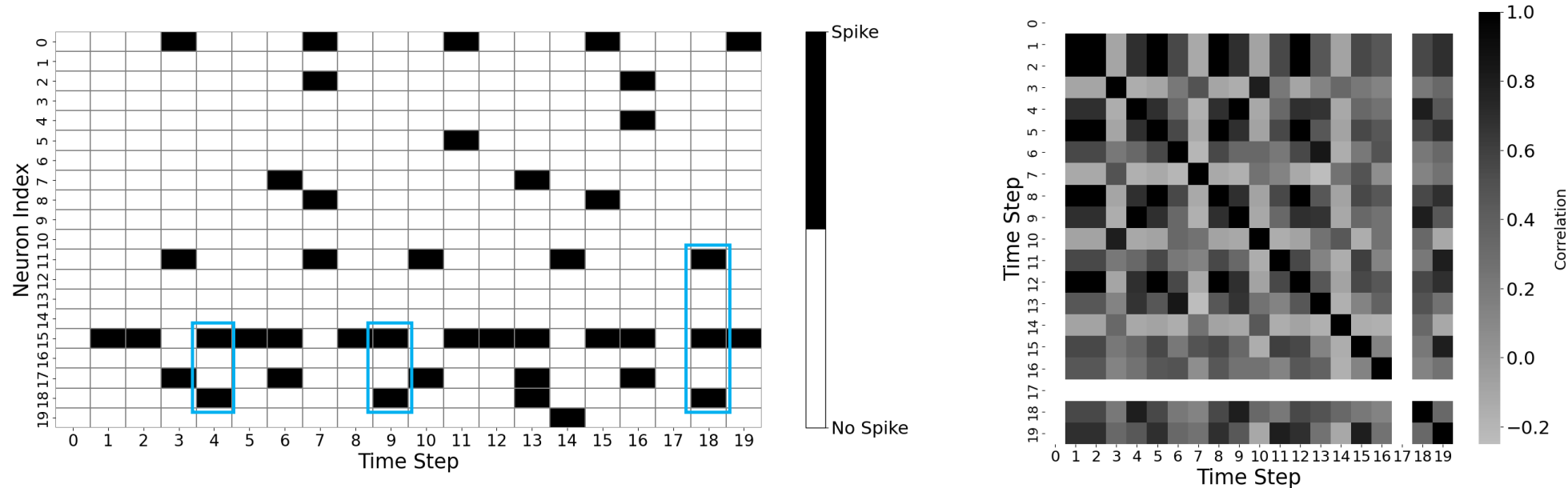


Fig. 5: Temporal redundancy in spikes of a 784-20-10 SNN trained via BPTT on MNIST.

Our Contributions

- **Frequent Spike Pattern Compression (FSPC):** A framework for compressing co-firing neuron activity in SNNs [5]
- Leverages pattern mining algorithms to identify neuron sets that frequently fire together
- Merges these co-firing sets to reduce overall spike data size

Content

- Background
- Motivation
- Proposed Method
 1. Pattern Finding Phase
 2. Compression Phase
- Results
- Conclusion

Proposed Method Overview

Key Steps:

1. Pattern Finding Phase
2. Compression Phase

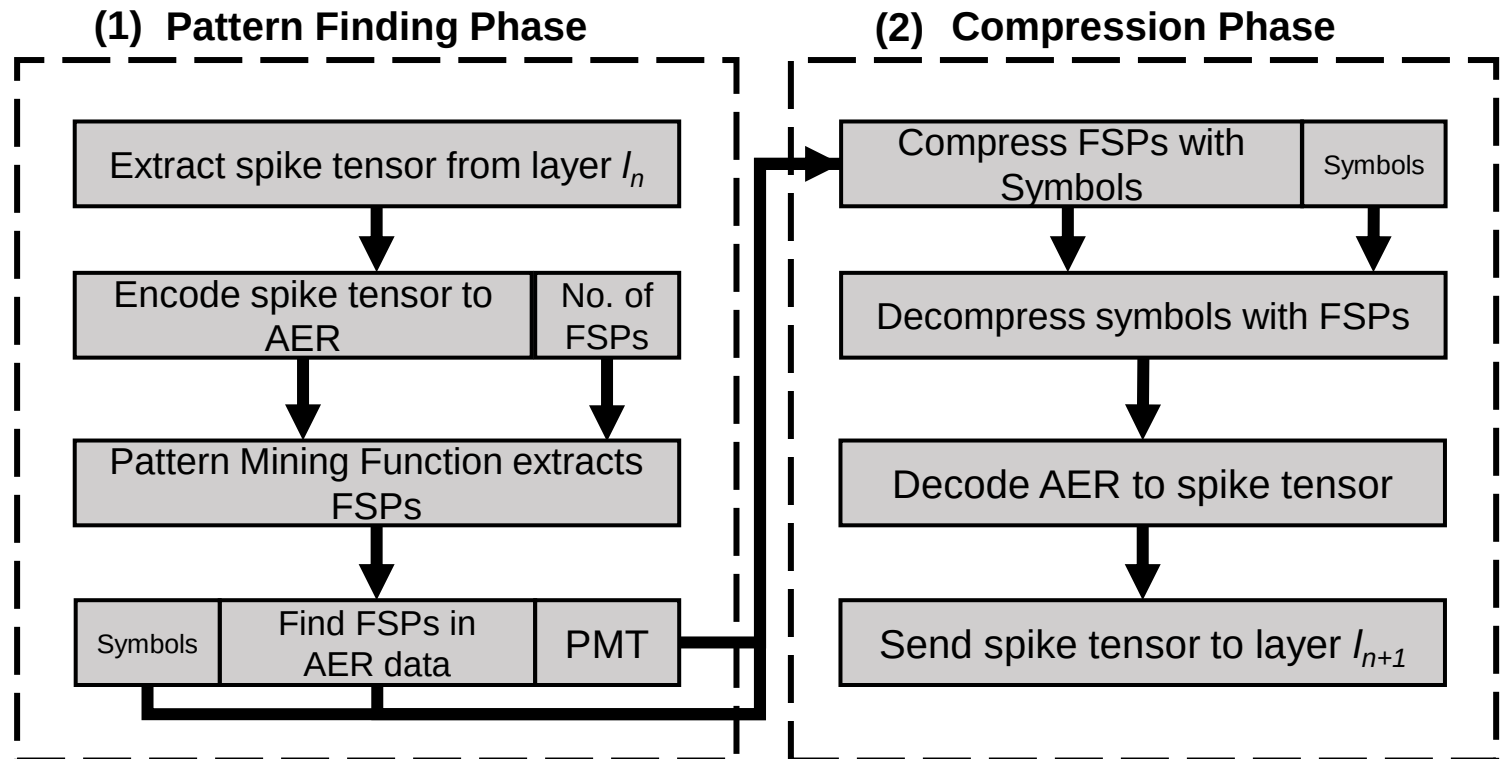


Fig. 6: FSPC Framework

Important Terms

- **Frequent Spike Pattern (FSP):** Set of neurons firing simultaneously at same timestep
- **Pattern Match Threshold (PMT):** Value (0–1) defining the minimum proportion of FSP neurons that must appear in AER data for a valid match
- The required number of FSP neurons in the AER data is:

$$N_{\text{required}} = \lceil \text{PMT} \times |\text{FSP Set}| \rceil$$

$|\text{FSP Set}|$ = number of neurons in FSP set

$\lceil \rceil$ = ceiling function

Location of FSPC in SNN

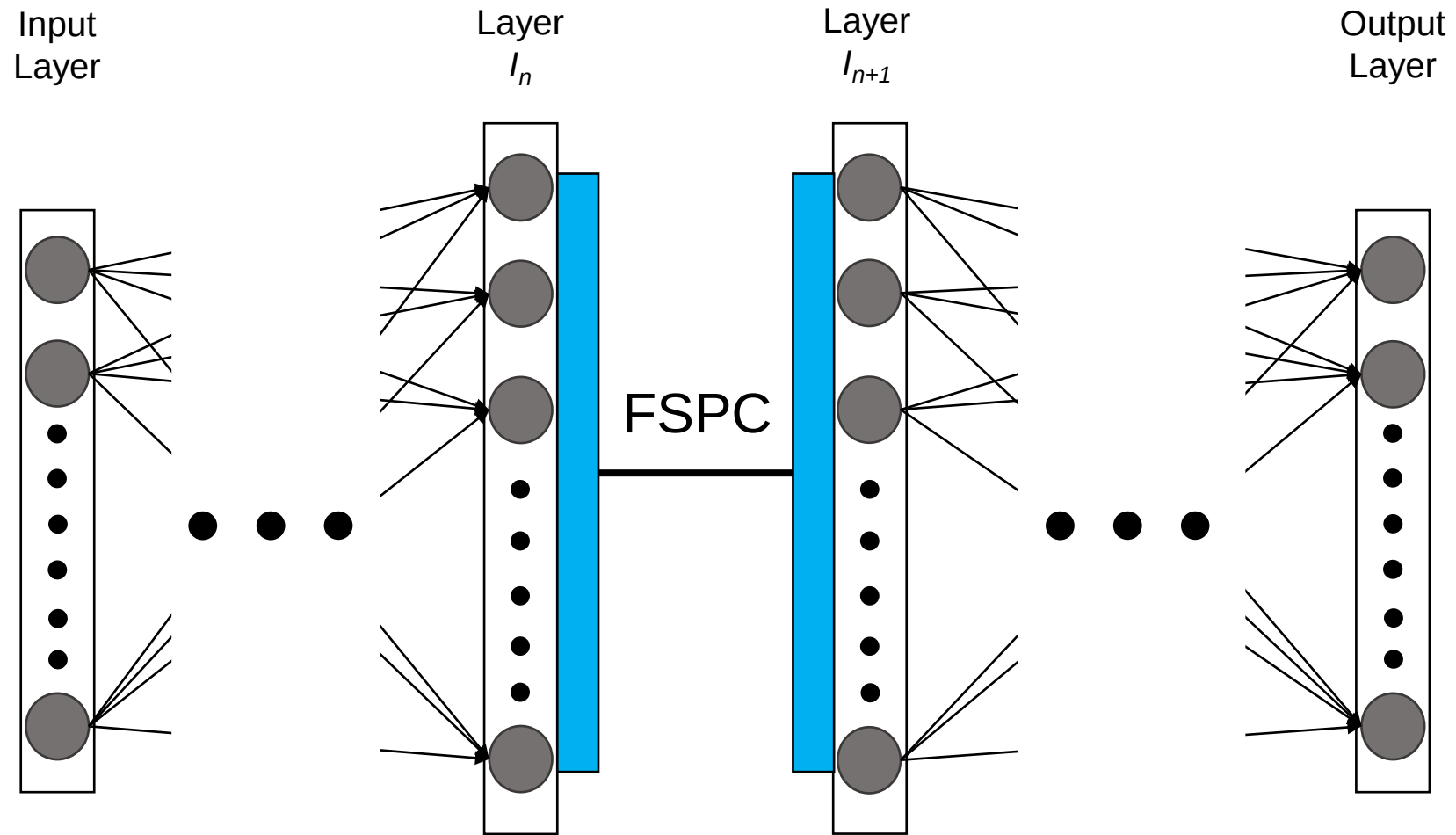


Fig. 7: Location of FSPC in SNN

(1) Pattern Finding Phase

Extract spike tensor from layer l_n and encode it to AER

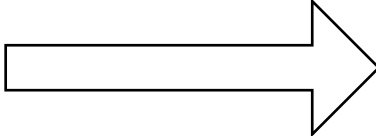
Spike Data

Neuron Index

	0	1	2	3	4	5	6	7
0	1	0	1	0	0	1	1	0
1	1	1	0	0	1	0	0	1
2	0	1	1	0	0	1	1	0
3	1	0	1	0	0	1	1	0
4	1	1	0	0	1	0	1	0
5	1	0	1	0	0	1	1	1
6	1	0	1	0	0	1	1	0
7	1	0	1	0	1	1	0	0

Time Step

Convert to
AER format



Encode to AER Data

	0	1	2	3	4	5	6	7
0	0		2			5	6	
1	0	1			4			7
2		1	2			5	6	
3	0		2			5	6	
4	0	1			4		6	
5	0		2			5	6	7
6	0		2			5	6	
7	0		2		4	5		

(1) Pattern Finding Phase

Extract FSPs using Pattern Mining Algorithms and assign them to symbols

Encoded AER Data

	0	1	2	3	4	5	6	7
0	0		2			5	6	
1	0	1			4			7
2		1	2			5	6	
3	0		2			5	6	
4	0	1			4		6	
5	0		2			5	6	7
6	0		2			5	6	
7	0		2		4	5		

Pattern Mining
Algorithms extract FSP

FSP = 2

Extracted FSPs are assigned to
symbols

$$\alpha_1 = \{0, 2, 5, 6\}$$

$$\alpha_2 = \{1, 4\}$$

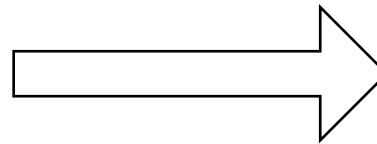
(1) Pattern Finding Phase

Find the presence of FSPs in AER data

Encoded AER Data

	0	1	2	3	4	5	6	7
0	0		2			5	6	
1	0	1			4			7
2		1	2			5	6	
3	0		2			5	6	
4	0	1			4		6	
5	0		2			5	6	7
6	0		2			5	6	
7	0		2		4	5		

Scanning AER for the
extracted FSPs



Find FSPs in AER

	0	1	2	3	4	5	6	7
0	0		2			5	6	
1	0	1			4			7
2		1	2			5	6	
3	0		2			5	6	
4	0	1			4		6	
5	0		2			5	6	7
6	0		2			5	6	
7	0		2		4	5		

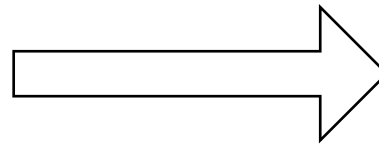
(2) Compression Phase

Compress FSPs with Symbols in AER data

FSPs in AER

	0	1	2	3	4	5	6	7
0	0		2			5	6	
1	0	1			4			7
2		1	2			5	6	
3	0		2			5	6	
4	0	1			4		6	
5	0		2			5	6	7
6	0		2			5	6	
7	0		2		4	5		

Replacing selected
FSPs with α_1 and α_2



Compress FSPs to Symbols

			0	1	2	3	4	5	6	7
0	α_1									
1		α_2	0							7
2				1	2			5	6	
3	α_1									
4		α_2	0						6	
5	α_1									7
6	α_1									
7			0		2		4	5		

Compressed FSPs

(2) Compression Phase

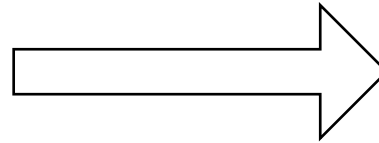
Decompress Symbols with FSP in AER data

Compressed Symbols

			0	1	2	3	4	5	6	7
0	α_1									
1		α_2	0							7
2				1	2			5	6	
3	α_1									
4		α_2	0						6	
5	α_1									7
6	α_1									
7			0		2		4	5		

Compressed FSPs

Replacing symbols α_1
and α_2 with their FSPs



Decompress Symbols to FSPs

			0	1	2	3	4	5	6	7
0	0		2					5	6	
1	0	1					4			7
2		1	2					5	6	
3	0		2					5	6	
4	0	1					4		6	
5	0		2					5	6	7
6	0		2					5	6	
7	0		2				4	5		

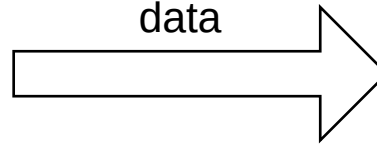
(2) Compression Phase

Decode recovered AER to spike tensor and send to layer l_{n+1}

AER Data

	0	1	2	3	4	5	6	7
0	0		2			5	6	
1	0	1			4			7
2		1	2			5	6	
3	0		2			5	6	
4	0	1			4		6	
5	0		2			5	6	7
6	0		2			5	6	
7	0		2		4	5		

Revert AER
data to spike
data



Decode to Spike Data

	0	1	2	3	4	5	6	7
0	1	0	1	0	0	1	1	0
1	1	1	0	0	1	0	0	1
2	0	1	1	0	0	1	1	0
3	1	0	1	0	0	1	1	0
4	1	1	0	0	1	0	1	0
5	1	0	1	0	0	1	1	1
6	1	0	1	0	0	1	1	0
7	1	0	1	0	1	1	0	0

Compression Types

$$\alpha_1 = \{0, 2, 5, 6\}$$

$$\alpha_2 = \{1, 4\}$$

Lossless
compression

PMT = 1

	0	1	2	3	4	5	6	7
0	0		2			5	6	
1	0	1			4			7
2		1	2			5	6	
3	0		2			5	6	
4	0	1			4		6	
5	0		2			5	6	7
6	0		2			5	6	
7	0		2		4	5		

Pattern at
timestep 2 and 7:
 $[0.7 \cdot 4] = 3$
elements of α_1
present

Lossy
compression

PMT = 0.7

	0	1	2	3	4	5	6	7
0	0		2			5	6	
1	0	1			4			7
2		1	2			5	6	
3	0		2			5	6	
4	0	1			4		6	
5	0		2			5	6	7
6	0		2			5	6	
7	0		2		4	5		

Content

- Background
- Motivation
- Proposed Method
- **Results**
- Conclusion

Evaluation and Result

- Model Configuration:

Model	Input Neurons (IN)	Hidden Neurons (HN)	Output Neurons (ON)
Three-layer	784	100	10
Four-layer	784	100 (HN1), 60 (HN2)	10

- Datasets Used: MNIST, Fashion-MNIST (3000 images out of 10000)
- Framework Used: snnTorch
- Pattern Mining Algorithms: mlxtend fpmax [6], PAMI's FP-growth and maxFP-growth [7]
- Minimum Support Value: 0.25 (0.35 for four-layer SNN in F-MNIST)
- Criteria: Evaluation based on PMT and FSP

Evaluation based on FSP

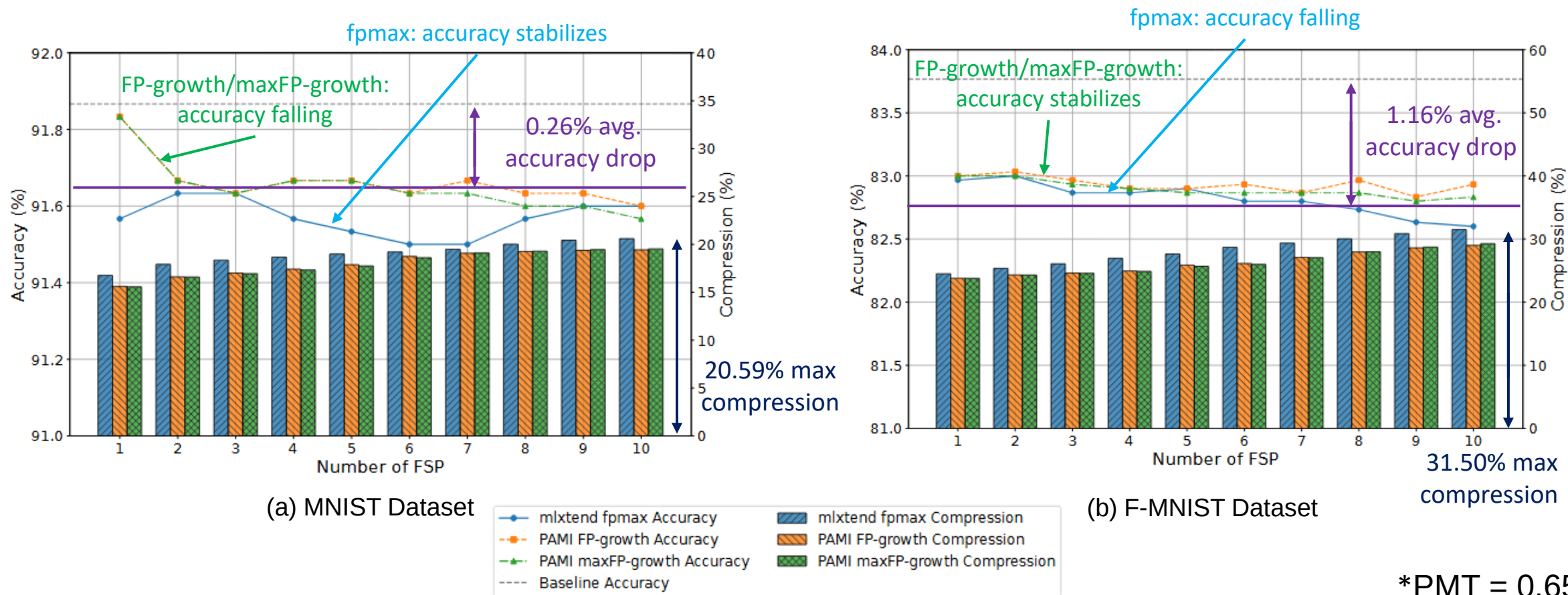


Fig. 8: Evaluation of FSPC between layer HN1–HN2 in a 784-100-60-10 neuron SNN with respect to FSP

Evaluation based on PMT

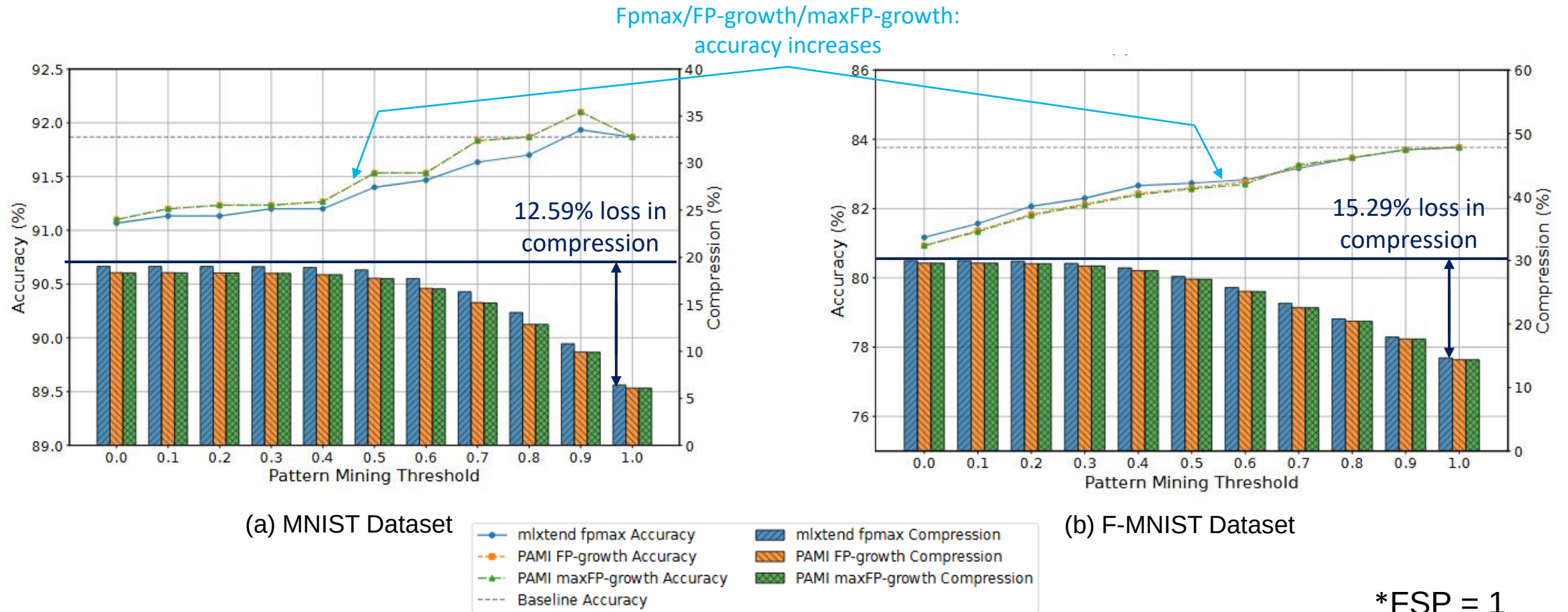


Fig. 9: Evaluation of FSPC between layer HN1–HN2 in a 784-100-60-10 neuron SNN with respect to PMT

Content

- Background
- Motivation
- Proposed Method
- Results
- Conclusion

Conclusion

- Lossy spike compression proposed for SNNs
- 12–15% spike-data reduction with 1–3% accuracy loss
- Can improve simulation efficiency and support adaptive, hardware-aware compression
- Future: Develop faster, more scalable algorithms and test on complex models and datasets

Publication & Code Availability

- This work was published at the 2025 IEEE MCSoc Conference under the title “**FSPC: A Lossy Spike Compression Through Correlated-AER Merging in Spiking Neural Networks**”
- Code implementation is available here: <https://github.com/klab-aizu/fspc>

References

- [1] W. Maass, “Networks of spiking neurons: The third generation of neural network models,” *Electron. Colloquium Comput. Complex.*, vol. TR96, 1996.
- [2] M. A. Mahowald, “Vlsi analogs of neuronal visual processing: A synthesis of form and function,” PhD dissertation, California Institute of Technology, Pasadena, CA, USA, May 1992
- [3] J. Han et al., “Mining frequent patterns without candidate generation,” in *ACM SIGMOD Conference*, 2000, pp. 1–12.
- [4] G. Grahne and J. Zhu, “High performance mining of maximal frequent itemsets,” in *Proceedings of the 6th SIAM International Workshop on High Performance Data Mining*, 2003, p. 34.
- [5] S. Ganesh, H. Yuga, Z. Wang, and K. N. Dang, “FSPC: A lossy spike compression through correlated-aer merging in spiking neural networks,” in *2025 IEEE 18th International Symposium on Embedded Multicore/Many-core Systems-on-Chip (MCSoc)*. IEEE, 2025, pp. 386–393.
- [6] S. Raschka, “Mlxtend: Providing machine learning and data science utilities and extensions to python’s scientific computing stack,” *The Journal of Open Source Software*, vol. 3, no. 24, Apr. 2018.
- [7] U. K. Rage et al., “PAMI: An open-source python library for pattern mining,” *Journal of Machine Learning Research*, vol. 25, no. 209, pp.1–6, 2024.

Thank you for your attention!